

SAM cours 3a

Adaptive Radix Tree

Ref biblio: ICDE 2013

The Adaptive Radix Tree: ARTful Indexing for Main-Memory Databases
by Viktor Leis, Alfons Kemper and Thomas Neumann

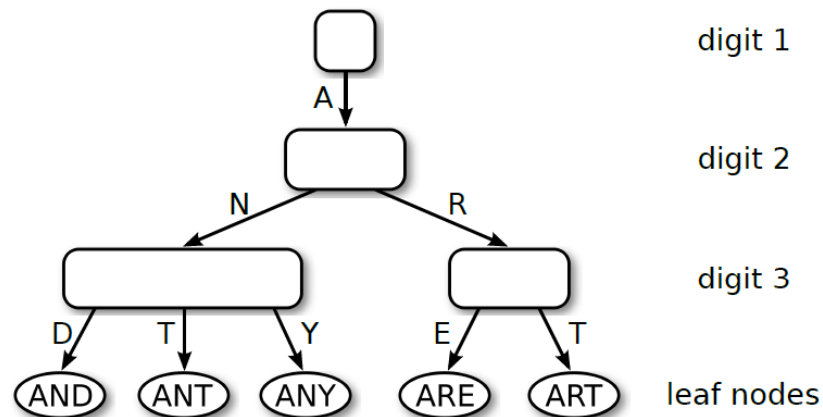
<https://db.in.tum.de/~leis/papers/ART.pdf>

Adaptive Radix Tree

- Dédié aux SGBD en mémoire centrale
 - DuckDB : fast in-process SQL OLAP database
 - https://duckdb.org/why_duckdb.html#duckdbisfast
- Cas d'usage
 - Index secondaires
- Constat
 - Les arbre B+ ne sont pas efficaces pour les SGBD en mémoire
 - Le temps de lecture d'une page n'est plus prépondérant
 - Il est du même ordre de grandeur que le temps de parcourir un nœud.
 - La majeure partie du temps est passée à rechercher une valeur dans un nœud...
 - Utilisation peu efficace de la mémoire et du cache de 1^{er} niveau

Adaptive Radix Tree

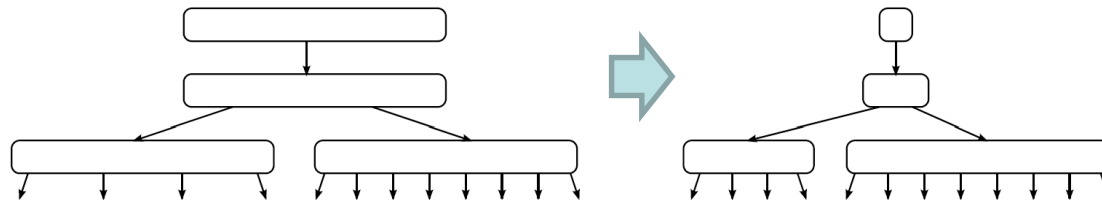
- Besoin
 - Index peu profond pour accès rapide à une feuille
 - Index compact : faible empreinte en mémoire
- Défis
 - mieux utiliser le cache du processeur ?
 - Un nœud doit tenir dans le cache L1
 - accès rapide à une valeur dans un nœud ?
 - indexer le contenu d'un nœud
- Idée : proposer plusieurs tailles de nœuds (small, medium, large, xl)
 - pour chaque taille, avoir un accès dédié très efficace



Exemple : la taille des nœuds est adaptée aux données

Choix de la taille d'un noeud

- Compromis entre 2 objectifs
 - Vitesse :
 - Grande taille = accès rapide car la hauteur sera moindre
 - Espace :
 - Petite taille = moins de cases vides dans les nœuds qui ont peu de fils
- Optimiser le compromis en adaptant la taille des noeuds



Propriétés

- Un nœud contient une partie de la clé
 - Clé composé de n morceaux
 - Nœud du niveau i : $i^{\text{ème}}$ morceau de la clé
- La hauteur de l'arbre
 - dépend seulement de la longueur de la clé : hauteur = n
 - ne dépend **pas** du nombre de clés
- Mise à jour efficace
 - Modification simple et rapide de la structure de l'arbre
 - Pas de fusion ni de redistribution de valeurs
 - contrairement aux arbres B+
 - Structure indépendante de l'ordre d'insertion
- Une branche forme une clé
 - Pas nécessaire de stocker les clés dans les feuilles (économie d'espace)
 - Les feuilles ne contiennent que les valeurs associées aux clés
- Les clés sont ordonnées

Découpage de la clé

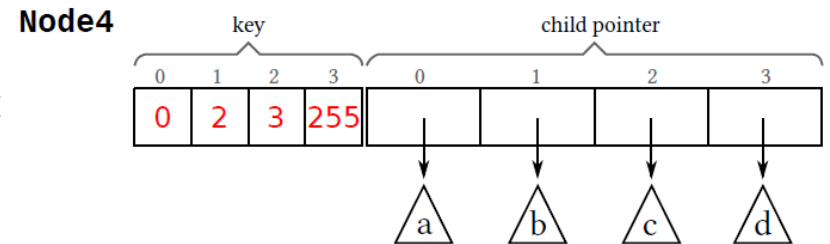
- La clé est découpée en **morceaux** de 1 octet
 - Exple: index sur le prénom
 - On considère que chaque lettre tient sur 1 octet : 1 **morceau** = 1 **lettre**
 - ALICE découpée en: A-L-I-C-E
 - Indexer : ALICE, ALFRED, ASTRID, BART, BILL, BILLIE, BOB
 - Niveau 1 = Racine = 1^{er} morceau: {A, B}
 - Niveau 2 = fils = 2^{ème} morceau: {L, S} {A, I, O}
 - Niveau 3 = petits fils = 3^{ème} morceau: {I, F} {T} {R} {L} {}
 -
- Pourquoi choisir une taille de 1 octet ?
 - Un morceau doit être un multiple d'un octet pour être traité rapidement par un processeur 64bits (mots de 8 octets)
 - Un morceau doit être assez « petit » pour représenter un petit ensemble de valeurs qu'on peut retrouver rapidement en utilisant un petit tableau
 - 1 octet a 256 valeurs possibles par morceau
 - mais avec 2 octets on aurait 64K valeurs : trop de valeurs pour une recherche très rapide

Types de nœuds

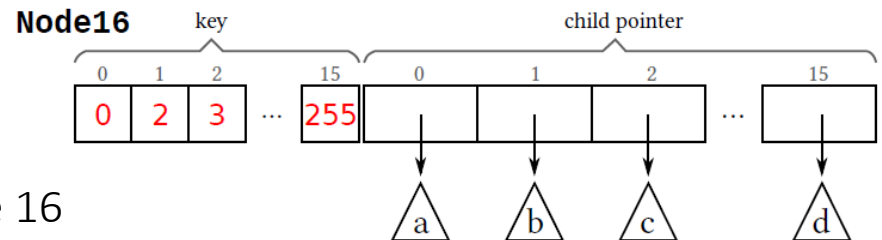
- Le nombre de *valeurs distinctes*, dans un morceau de clé, dépend des données
 - Données réelles : distribution irrégulière des valeurs
 - Présence de morceaux avec peu de valeurs, bcp de valeurs
- Plusieurs types de nœud sont définis. Ils sont *adaptés* au nombre de valeurs à indexer
 - Peu de valeurs
 - Structure compacte (taille proche du nombre de valeurs)
 - voir : Node4 et Node16
 - Beaucoup de valeurs
 - Accès rapide à la $n^{\text{ième}}$ valeur d'une clé
 - voir : Node48 et Node256

Noeud interne : Node 4 et Node16

- Node4
 - Un tableau de 4 clés
 - (exemple : 0, 2, 3 et 255)
 - Un tableau de 4 pointeurs
 - Rechercher une clé en temps constant
 - Exple pour rechercher la valeur 3
 - $[0, 2, 3, 255] == 3 \rightarrow [0, 0, 1, 0]$
 - $[0,0,1,0] \& [a,b,c,d] \rightarrow c$
 - recherche en seulement 2 instructions

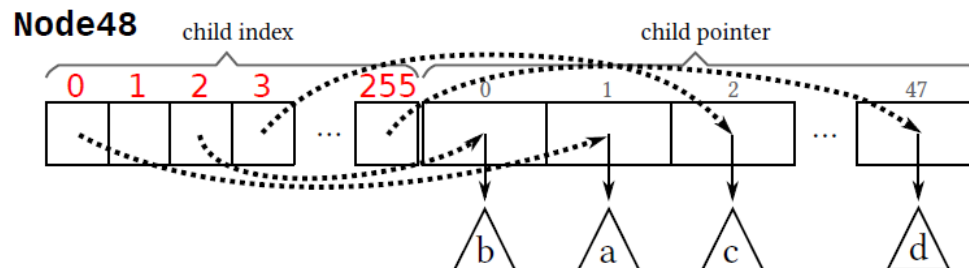


- Node16
 - idem mais avec des tableaux de taille 16
 - Les CPU modernes peuvent tester les 16 cases en un seul cycle CPU (Single Instruction on Multiple Data)



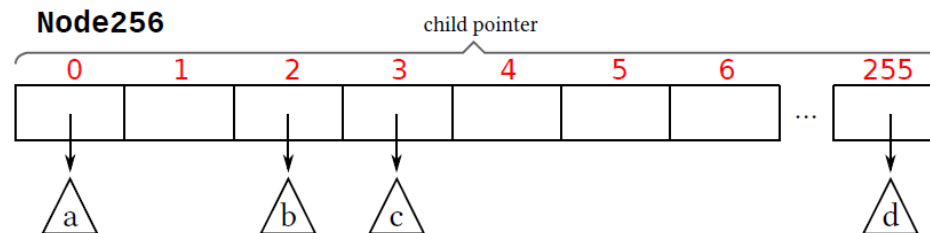
Noeud interne : Node48

- Ajouter un « mini index » des valeurs de clés
 - Tableau avec 255 cases pouvant contenir des *positions* de 0 à 47
 - La $i^{\text{ème}}$ case contient l'indice de l'adresse du fils associé à la clé i
 - Puis accès au tableau principal des adresses des fils
 - Exemple : retrouver la clé 3
 - $p = \text{index}[3]$ *p vaut 2*
 - $\text{child} = \text{children}[p]$ *child vaut c*



Nœud interne : Node256

- Accès direct au tableau contenant les adresses des fils
 - La $i^{\text{ème}}$ case contient l'adresse du fils associé à la clé « i »
- Exemple pour la clé 3 :
 - $\text{child} = \text{children}[3]$ *child vaut c*



Types de feuilles

- Une feuille contient la **valeur** associée à une clé
 - La clé est représentée par le chemin racine → feuille
 - la **valeur** est :
 - un ROWID dans le cas d'un index secondaires
 - un tuple dans le cas d'un index plaçant

3 types de feuilles sont définis :

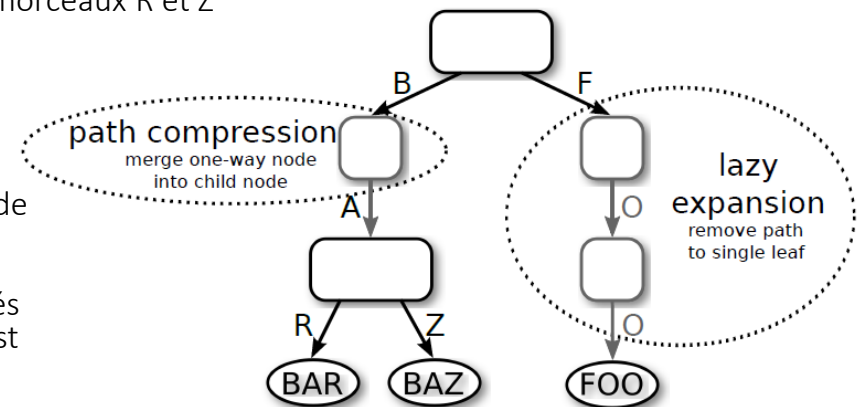
- Type de feuille le plus général
 - Contient une seule valeur
 - feuille placée à *l'extrémité* de chaque chemin
 - les tailles des **clés** et des **valeurs** peuvent **varier**
 - Inconvénient : augmente de 1 la longueur de tous les chemins, donc ralentit l'accès aux **valeurs** recherchées

Types de feuilles (suite)

- Feuille dédiée au stockage de plusieurs valeurs
 - Mêmes structures que les nœuds internes : 4 tailles possibles.
 - Toutes les **clés** doivent avoir la même taille
 - Associe le « dernier morceau » de la clé avec les **valeurs**
 - Toutes les feuilles sont dans le même niveau l'arbre (équilibré)
 - la taille des **valeurs** est fixe
 - mais elle peut être supérieure à la taille d'un pointeur
- Feuille mixte
 - Nœud interne mais avec emplacement mixte pointeur/valeur
 - Marqueur booléen : 1 si l'emplacement contient une **valeur**
 - La taille des **clés** peut varier
 - La taille des **valeurs** est constante = taille d'un pointeur
 - C'est la **meilleure option** pour les **index secondaires**
 - car la valeur est ROWID qui est un pointeur

Compacter l'arbre

- L'arité (nombre de fils) d'un nœud dépend des données
 - C'est le nombre de valeurs distinctes de la clé parmi les tuples ayant le même préfixe
 - Eviter d'avoir des nœuds avec un seul fils (nœud unaire)
 - Eviter de créer de tels nœuds → lazy expansion
 - Supprimer de tels nœuds → path compression
 - Nécessite de pouvoir **retrouver la clé** dans tuple indexé
- Lazy expansion
 - Ne pas créer des chemins de nœuds unaires atteignant une seule feuille.
 - On peut retrouver la feuille FOO sans utiliser les *derniers morceaux* de la clé (O, O).
- Path compression
 - Enlever un nœud unaire ayant un fils n-aire ($n > 1$)
 - **Comprimer** (=conserver) dans le fils le morceau de clé venant du nœud supprimé
 - A est « comprimé » dans le nœud indexant les morceaux R et Z



Rmq: Path compression « optimisée » tout en gardant des nœuds de taille préfixée.

- **Comprimer** jusqu'à 8 morceaux de clé au maximum (8 octets)
- Ensuite, mémoriser seulement le nombre de morceaux enlevés
- Nécessite de vérifier, quand on a atteint la feuille, que la clé est bien celle recherchée

Insertion d'une entrée

- Insérer un couple (clé, valeur) = insérer une feuille
 - Si la valeur est un ROWID, elle est insérée en tant que « valeur feuille » dans un nœud mixte
 - Sinon un nœud feuille contenant la valeur est créé et inséré dans l'arbre
- Tenir compte des règles de *lazy expansion* et *path compression*
- Traverser l'arbre pour atteindre le nœud dans lequel insérer la valeur
 - Si le nœud est une feuille
 - Peut se produire à cause de la stratégie *lazy expansion*
 - La feuille existante doit « descendre » d'un niveau, dans un nouveau nœud Node4
 - Déterminer la partie commune entre la clé existante et la nouvelle clé
 - Comprimer cette partie dans le nouveau nœud (path compression)
 - Insérer la valeur dans le nouveau nœud
 - La stratégie *lazy expansion* garantit qu'un seul nœud est créé
 - Si le nœud est plein, le remplacer par un type plus grand, puis insérer la valeur
 - Si la clé à insérer est différente de la partie comprimée dans le nœud
 - Peut se produire à cause de la stratégie *path compression*
 - Le nœud doit « descendre » d'un niveau, dans un nouveau nœud Node4
 - Ajuster la partie « comprimée » de la clé conservée dans le nœud

Suppression d'une entrée

- Réduire la taille du nœud
 - Garantir qu'un nœud de type nodeN a toujours au moins N valeurs
- Comprimer un chemin si un nœud n'a plus qu'un seul fils
 - Voir path compression

Création de l'index

- Méthode dite *bulk load*
 - Lorsque la table à indexer contient déjà bcp de tuples
 - Ne **pas** insérer les tuples un par un dans l'index
 - Méthode rapide car on peut déterminer à l'avance la taille et le contenu de chaque nœud de l'index
 - Approche Top-down
 - **partitionner** les clés en se basant sur le 1^{er} morceau
 - Créer le nœud de niveau 1 (racine)
 - Recommencer pour chaque partition récursivement
 - Créer les nœuds de niveau 2, ..., n

Taille d'un noeud

- Tailles mesurées en octets
- Adresse d'un nœud : 8 octets (si processeur 64b)
- Pour tous les types de nœud
 - Entête (16)
 - Espace pour conserver les éventuels morceaux de clé «compressés» (8 octets)
 - Type de nœud, nombre fils (non nuls)
 - Tableau des fils : $8 * \text{nombre de fils}$
- Node4
 - Entête (16) + Index (4) + tableau ($4*8$) = 52
- Node16
 - Entête (16) + Index (16) + tableau ($16*8$) = 160
- Node48
 - Entête (16) + Index (256) + tableau ($48*8$) = 656
- Node256
 - Entête (16) + tableau ($256*8$) = 2064
- Nœuds mixtes
 - Ajouter le tableau des marqueurs booléens valeur/nœud (8 marqueurs par octet)
 - Taille : $4/8 = 0.5$ soit 1 octet pour Node 4, ..., $256/8 = 32$ pour Node256

Taille de l'index

- Taille maximale d'un arbre ?
 - Soit C le nombre de clés indexées
 - On considère le cas le plus défavorable
 - Arbre ayant le nombre maximal de nœuds
 - Feuilles de type Node4 avec seulement 2 valeurs
 - On aura $C/2$ nœuds contenant des valeurs
 - *Borne supérieure théorique*
 - Le nombre N de nœuds vaut :
 $N = C/2$ (feuilles) + $C/4$ (parents des feuilles) + ... + $2 + 1$ (racine)
on pose $C/2 = 2^{p-1} \Leftrightarrow C = 2^p$
 $N = 1 + 2 + \dots + 2^{p-1} = 2^p - 1$ (somme des termes d'une suite géo) = $C - 1$
 $N < C$
 - Taille arbre < 52 * C
 - Pour comparaison, les solutions connexes ont une borne soit infinie soit indéfinie
 - Taille réelle en pratique
 - Arbre encore plus compact avec une taille d'environ $8 * C$

Tri des clés

- L'arbre doit conserver l'ordre des clés
 - L'encodage binaire d'une clé doit conserver l'ordre alphanumérique de l'attribut indexé
- Transformations existantes pour
 - les caractères, les nombres
- Valeur NULL
 - Pour un attribut dont la taille fait n octets
 - La valeur minimale du domaine est une série de n octets 0
 - Ajouter une valeur inférieure à toutes les autres
 - Valeur de taille $n+1$: série de $(n+1)$ octets 0
 - Ré-encoder la valeur minimale du domaine sur $n+1$ octets
 - série de n octets 0 suivie de l'octet 1
 - Avantage: aucun ré-encodage pour les autres valeurs

Exercice sur les index ART

- Insertion pour une situation de
 - lazy extension
 - path compression
- Accès par intervalle
 - Retrouver les clés comprises dans $[a,b]$
 - a et b peuvent ne pas exister
 - Détailler l'accès à la plus petite clé supérieure à a

Fiche de lecture

- Indexer des attribut textuels ?
- Index unique versus non unique ?
- Parcours latéral : chainage des feuilles ?
- Index composé ?